Disk Stacking

AlgoExpert · Hard · Dynamic Programming

The problem

- Disks [width, depth, height]; stack them so each disk is strictly smaller in all three dimensions than the one below, maximising total height.
- Returns the chosen disks ordered from top (smallest) to bottom (largest).

Intuition

- Sort disks by height; each disk's best stack is itself on top of the best compatible stack below.

Brute force vs optimal

- Try all subsets/orders → exponential
- Sort by height, LIS-style DP → $O(n^2)$
- Back-pointers rebuild the stack

Approach

- 1. Sort by height
- 2. For each disk, check smaller earlier disks
- 3. Keep the best height and a predecessor link
- 4. Rebuild the stack

Worked example

w	d	h	best height
1	3	1	1
2	1	2	
3	2	3	
2	3	4	
4	4	5	
2	2	8	

```
disk [1, 3, 1]: best stack height 1
```

Disk [1, 3, 1]: 1

Worked example

w	d	h	best height
1	3	1	1
2	1	2	2
3	2	3	
2	3	4	
4	4	5	
2	2	8	

```
disk [2, 1, 2]: best stack height 2
```

Disk [2, 1, 2]: 2

Worked example

w	d	h	best height
1	3	1	1
2	1	2	2
3	2	3	5
2	3	4	
4	4	5	
2	2	8	

```
disk [3, 2, 3]: best stack height 5
```

Disk [3, 2, 3]: 5

Worked example

w	d	h	best height
1	3	1	1
2	1	2	2
3	2	3	5
2	3	4	4
4	4	5	10
2	2	8	

```
disk [4, 4, 5]: best stack height 10
```

Disk [4, 4, 5]: 10

Worked example

w	d	h	best height
1	3	1	1
2	1	2	2
3	2	3	5
2	3	4	4
4	4	5	10
2	2	8	8

```
disk [2, 2, 8]: best stack height 8
```

Disk [2, 2, 8]: 8

Worked example

```
stack: [[2, 1, 2], [3, 2, 3], [4, 4, 5]]

height 10
```

Best stack height 10

Java solution

```java
public class DiskStacking {
    public static List<Integer[]> diskStacking(List<Integer[]> disks) {
        List<Integer[]> d = new ArrayList<>(disks);
        d.sort(Comparator.comparingInt(x -> x[2]));
        int n = d.size(), best = 0;
        int[] h = new int[n], prev = new int[n];
        for (int i = 0; i < n; i++) {
            h[i] = d.get(i)[2];
            prev[i] = -1;
            for (int j = 0; j < i; j++) {
                Integer[] a = d.get(j), b = d.get(i);
                if (a[0] < b[0] && a[1] < b[1] && a[2] < b[2] && h[j] + b[2] > h[i]) { h[i] = h[j] + b[2]; prev[i] = j; }
            }
            if (h[i] > h[best]) best = i;
        }
        LinkedList<Integer[]> out = new LinkedList<>();
        for (int i = best; i != -1; i = prev[i]) out.addFirst(d.get(i));
        return out;
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Edge cases & pitfalls

- All three dimensions strictly smaller
- Sorting by one dimension is enough
- Track prev to rebuild

Interview tips & follow-ups

- Relate to Box Stacking / LIS

Recap

- Sort disks by height; each disk's best stack is itself on top of the best compatible stack below.
- Complexity: O(n^2)